EMORY UNIVERSITY ILLINOIS INSTITUTE OF TECHNOLOGY UNIVERSITY OF NOTRE DAME

Scaling Teams or Scaling Time? Memory Enabled Lifelong Learning in LLM Multi-Agent Systems

Shanglin Wu[1], **Yueqing Liang**[2], **Yuyang Luo**[1], **Ken Su**[1], **Kaiwen Shi**[3] **and Kai Shu**[1]
[1]Emory University, [2]Illinois Institute of Technology, [3]University of Notre Dame

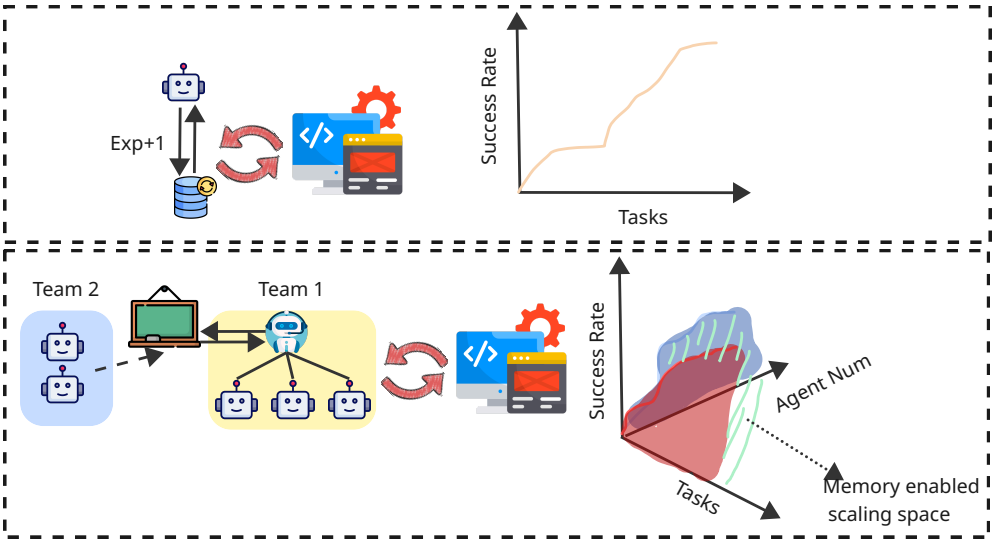

Figure 1

1. Introduction

Large language model (LLM)-based agent systems have demonstrated remarkable capabilities across diverse domains, from complex reasoning and code generation to embodied task execution and scientific discovery (Chen et al., 2024b; Wu et al., 2024). These systems extend beyond passive language generation to exhibit autonomous decision-making, tool use, and environmental interaction. A natural extension of this paradigm involves scaling from individual agents to multi-agent systems (MAS), where multiple LLM agents collaborate to tackle problems that exceed the capacity of any single agent. Such collaborative architectures prove essential for tasks requiring diverse expertise, extended context processing, parallel execution, or collective verification to reduce hallucinations (Chen et al., 2024a; Du et al., 2023). Beyond scaling team size, a distinct dimension of scaling emerges at lifelong learning (Zheng et al., 2025), where agents continuously adapt and improve over time.

Current research predominantly addresses these two scaling dimensions in isolation. Studies on team size scaling investigate how performance varies with the number of agents, revealing that collaborative scaling follows logistic growth patterns (Qian et al., 2024) and smaller models with more agents can outperform larger models with fewer agents (Li et al., 2024). Separately, research on life-long learning focuses on how individual agents improve through accumulated experience,

Corresponding author(s): kai.shu@emory.edu

developing sophisticated memory architectures that enable continuous adaptation (Zheng et al., 2025). However, the intersection of these dimensions remains largely unexplored: while we understand how adding more agents affects performance, and separately how individual agents can improve through accumulating experience, we lack systematic understanding of how team size influence the lifelong learning ability of a LLM agent system. When time size increasing, the lifelong learning behavior may highly influenced by communiation cost and information fragmentation (Kim et al., 2025). It introduces challenge on memory management when the scale exponential growth. This consideration reveals a new perspective of trade-off: should we choose a larger MAS or should we choose a system with better lifelong learning ability? Moreover, we argue that investigating both dimensions jointly is essential because real-world deployments require cost control whose solution is essentially determined by the trade off between team size and life long learning ability.

Before investigating the correlation between lifelong learning ability and team size, we must first examine what factors govern a system's lifelong learning capability which is a crucial question that lacks systematic understanding. Among the many potential factors, we focus on memory as the primary element governing lifelong learning ability, as memory serves as the fundamental mechanism through which agents retain, retrieve, and build upon past experiences (Zhang et al., 2025; Zheng et al., 2025). The challenge of memory management scales with memory complexity, which Kim et al. (2025) have demonstrated depends on both team size and communication mechanisms. Moreover, we argue that memory topology which determines who can read or write which parts of memory also emerges as another crucial factor that determines memory complexity. Even under identical communication mechanisms, the complexity distribution across individual modules varies substantially with different topological configurations, as shown in figure 2. This leads to our first research question **RQ1: How should we get better lifelong learning ability via better memory module designing?** Notice, there are still lack of works focus on lifelong learning ability(previous design may still fail)

To address these research question, we propose a novel multi-agent memory framework **LLMA-Mem**. We conduct comprehensive experiments across three representative memory topology configurations as demonstrated in figure 2: (1) *local memory*, where each agent maintains an isolated memory store with no cross-agent knowledge sharing; (2) *shared memory*, where all agents access a centralized collective memory pool; and (3) *hybrid memory*, combining local memories with controlled access to shared knowledge repositories. We evaluate these configurations across varying team sizes to characterize the landscape. Crucially, our evaluation requires benchmarks that go beyond traditional static question answering or single turn reasoning tasks. We argue that suitable benchmarks for studying lifelong learning in multi-agent systems must satisfy several criteria: (i) tasks should require multi-step agentic reasoning with environmental feedback; (ii) task sequences should exhibit temporal dependencies where past experience can inform future performance; (iii) the benchmark should support variable team configurations; and (iv) performance gains from accumulated experience should be measurable and distinguishable from in-context learning effects. Based on these criteria, we select MultiAgentBench (Zhu et al., 2025) as our primary evaluation testbeds.

RQ2: How does lifelong learning ability interact with team size in these systems? To answer **RQ2**, we study *team-evolving* multi-agent systems over long task sequences and explicitly quantify how increasing team size changes the marginal value of experience. Intuitively, larger teams can provide stronger parallelism, which may accelerate the accumulation of useful knowledge early on. At the same time, scaling the team introduces coordination overhead, duplicated effort, and information fragmentation, potentially making it harder to consolidate learning into stable memories and thereby weakening long-horizon gains. This suggests that lifelong learning is not monotonic in team size, but is governed by a trade-off between scaling team and scaling time. We operationalize lifelong learning ability as the performance improvement over time under controlled conditions and analyze how learning curves, memory growth, and resource cost shift as team size increases across different

memory topologies. The result is a two-dimensional scaling landscape that reveals when adding agents amplifies learning, when it suppresses learning due to coordination costs, and which memory designs best preserve long-term improvement as systems scale.

Through systematic analysis, we demonstrate how the interplay between memory topology and team size creates a nuanced landscape for agentic scaling, offering guidance for researchers and practitioners in designing effective multi-agent systems. Our main contributions are as follows:

- We formally characterize the intersection of team size scaling and lifelong learning scaling, introducing the concept of **team-evolving systems** where agent collectives develop improved coordination and shared knowledge over successive tasks.
- We conduct the first systematic empirical study examining how three representative memory topologies interact with team size to influence lifelong learning performance in multi-agent LLM systems.
- We identify critical trade-offs between team size and lifelong learning ability, providing practical guidelines for when to scale horizontally (more agents) versus temporally (longer learning horizons) under resource constraints.
- We release our experimental framework and benchmark adaptations to facilitate future research on memory-aware multi-agent system design.

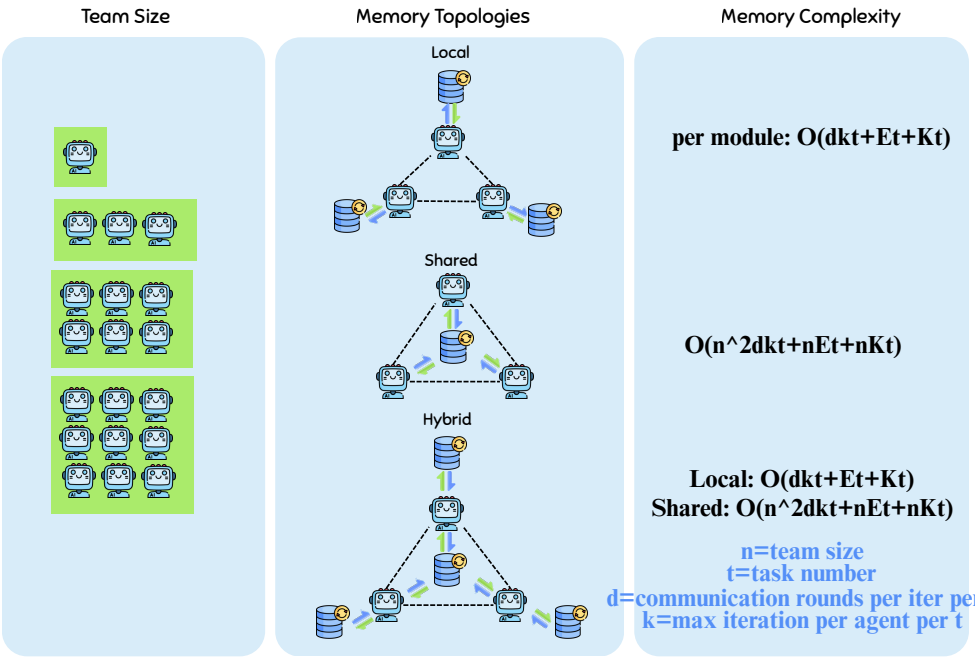

Figure 2

2. Related Works

2.1. Scaling of team size in LLM Multi-agent Systems

2.2. Lifelong learning in LLM-based Agent Systems

2.3. Memory in LLM-based Agent Systems

Memory mechanisms have become foundational to LLM-based agents, enabling them to transcend the stateless nature of base language models and develop persistent, adaptive behaviors. Early influential work by Park et al. (2023) introduced Generative Agents, which maintain a memory stream of experiences, synthesize higher-level reflections, and dynamically retrieve relevant memories for planning—demonstrating emergent social behaviors in a 25-agent sandbox environment. MemGPT (Packer et al., 2023) proposed a hierarchical two-tier architecture inspired by operating system virtual memory, with a main context analogous to RAM and external storage analogous to disk, enabling agents to self-edit their memory through function calls. More recent work like A-MEM (Xu et al., 2025) introduces dynamic memory organization with interconnected knowledge networks

The extension of memory to multi-agent systems introduces unique challenges distinct from single-agent memory (Wu and Shu, 2025). Transactive memory systems become essential for efficient task allocation (Zhang et al., 2025). Memory topology emerges as a critical design dimension, with three primary configurations identified in the literature: per-agent local memory where each agent maintains private stores, centralized shared memory using blackboard-style architectures, and hybrid approaches combining local perceptual memory with shared summarized world-state (Wu and Shu, 2025). Collaborative Memory frameworks (Rajaram and Pereira-Pasarin, 2010) formalize this with two-tier architectures featuring private fragments and shared fragments governed by dynamic access controls. LEGOMem (Han et al., 2025) specifically addresses procedural memory in multi-agent workflows, with orchestrator memory for high-level planning and fine-grained subtask retrieval for worker agents. Despite these advances, current multi-agent memory research primarily focuses on coordination efficiency within individual tasks rather than cross-task evolution—leaving substantial gaps in understanding how memory architectures enable teams to improve over successive experiences.

3. Method

3.1. LLMA-Mem (LifeLong Multi-Agent Memory)

We propose LLMA-Mem (LifeLong MultiAgent Memory), a unified memory framework designed to support lifelong learning across diverse multi-agent topologies. Unlike prior work that focuses on single-task coordination or individual agent memory, LLMA-Mem explicitly addresses how memory architectures enable agent collectives to improve through accumulated experience while scaling team size. The framework consists of three interdependent memory modules: episodic, procedural, and team state. Each serving distinct but complementary roles in enabling lifelong learning.

3.1.1. Memory Architecture

Episodic Memory Episodic memory stores comprehensive records of individual task experiences, preserving the full context necessary for pattern extraction and reflection. Each episode captures not only actions and outcomes but also the environmental state and team dynamics at the time of execution. This rich contextual information enables the system to ground procedural knowledge in concrete experiences and supports case-based reasoning when facing novel situations similar to past encounters.

Formally, each episode e_i is represented as:

$$e_i = \langle a_{\text{id}}, t, \tau, C, \mathcal{A}, o, c, \mathcal{L}, \mathcal{P}_{\text{rel}} \rangle \tag{1}$$

where a_{id} denotes the primary agent identifier, t is the timestamp, τ represents the task description, C encodes the team composition, $\mathcal{A}$ is the action sequence, o captures the outcome (success/failure with associated metrics), c stores the environmental context and state, $\mathcal{L}$ contains extracted lessons or insights, and $\mathcal{P}_{\text{rel}}$ maintains links to related procedural knowledge.

The episodic memory schema is structured as:

Listing 1 | Episodic Memory Schema

```
{
  "agent_id": str,
  "episode_id": str,
  "timestamp": datetime,
  "task_description": str,
  "team_composition": List[str],  # Agent IDs
  "actions_taken": List[Action],  # Sequence
  "outcome": {
    "success": bool,
    "metrics": Dict[str, float],
    "error_info": Optional[str]
  },
  "context_state": Dict,  # Environmental conditions
  "lessons_learned": List[str],  # Extracted insights
  "related_procedures": List[str],  # Procedure IDs
  "collaboration_quality": float  # Team performance
}
```

Episodic memory serves as the foundational layer from which procedural knowledge emerges. By maintaining the complete experiential history, the system can retrospectively analyze patterns, identify successful strategies under specific conditions, and refine its understanding of when particular approaches are applicable.

Procedural Memory Procedural memory abstracts reusable strategies, skills, and domain-specific procedures from episodic experiences. Unlike raw episodes, procedural knowledge represents generalized patterns that have demonstrated reliability across multiple instances. This generalization enables efficient knowledge transfer and reduces the computational cost of retrieval by distilling experience into actionable templates.

Each procedure p_j is defined as:

$$p_j = \langle a_{\text{id}}, t_c, t_u, \text{title}, \text{type}, \kappa, \pi, s, f, \mathcal{E}_{\text{src}}, \sigma \rangle \tag{2}$$

where a_{id} identifies the creator or owner, t_c and t_u are creation and last update timestamps, title and type categorize the procedure, κ encodes the knowledge content, π specifies prerequisites for applicability, s and f track success and failure counts, $\mathcal{E}_{\text{src}}$ links to source episodes, and σ represents a confidence score.

The procedural memory schema is:

Listing 2 | Procedural Memory Schema

```
{
  "agent_id": str,  # Or "shared" for shared memory
  "procedure_id": str,
  "timestamp_created": datetime,
  "timestamp_updated": datetime,
  "title": str,
  "type": str,  # Strategy/Skill/Algorithm/Heuristic
  "knowledge_content": str,
  "prerequisites": List[str],  # Applicability conditions
  "success_count": int,
  "failure_count": int,
  "success_rate": float,  # success/(success+failure)
  "avg_performance_gain": float,
  "source_episodes": List[str],  # Episode IDs
  "confidence_score": float,  # [0,1]
  "last_used": datetime,
  "viewed_times": int
}
```

The success rate ρ_j for procedure p_j is computed as:

$$\rho_j = \frac{s_j}{s_j + f_j + \epsilon} \tag{3}$$

where s_j and f_j are success and failure counts respectively, and ϵ is a small constant to prevent division by zero for unused procedures.

Procedural memory accumulates through a consolidation process that identifies recurring patterns in episodic memory. When a strategy or approach succeeds across multiple episodes (threshold $\theta_{\text{extract}} = 3$ successful instances), it is crystallized into a procedure. This extraction process transforms implicit, context-dependent knowledge into explicit, transferable procedures that can guide future actions.

Transactive Memory Transactive memory captures the evolving capabilities and collaboration dynamics of the multi-agent system. This component addresses the unique requirements of multi-agent lifelong learning by tracking not only individual agent competencies but also the collective patterns that emerge from repeated collaboration. As team size scales, understanding agent specializations and effective team configurations becomes critical for efficient task allocation and coordination.

The team state memory consists of two primary components: individual agent profiles and team collaboration patterns.

Agent profile α_k for agent k is represented as:

$$\alpha_k = \langle a_{\text{id}}, \mathcal{S}, \Gamma, \mathcal{H}, \xi \rangle \tag{4}$$

where a_{id} is the agent identifier, $\mathcal{S}$ represents learned specialization areas, Γ maps task types to proficiency scores, $\mathcal{H}$ encodes collaboration history with other agents, and ξ is a reliability score.

Team pattern ϕ_m for configuration m is defined as:

$$\phi_m = \langle C_m, \mathcal{T}_{\text{suited}}, \bar{\rho}_m, \omega_m \rangle \tag{5}$$

where C_m specifies the team composition, $\mathcal{T}_{\text{suited}}$ lists task types this configuration excels at, $\bar{\rho}_m$ is the average performance metric, and ω_m quantifies communication overhead.

The complete team state memory schema is:

Listing 3 | Team State Memory Schema

```
{
  "agent_profiles": {
    agent_id: {
      "specialization_areas": List[str],
      "skill_levels": Dict[str, float],  # task_type: proficiency
      "collaboration_history": {
        partner_id: {
          "interaction_count": int,
          "success_rate": float,
          "avg_communication_cost": float
        }
      },
      "reliability_score": float,
      "total_tasks_completed": int
    }
  },
  "team_patterns": {
    config_id: {
      "team_composition": List[str],  # Agent IDs
      "task_types_suited_for": List[str],
      "avg_performance": float,
      "communication_overhead": float,
      "usage_count": int
    }
  }
}
```

Team state memory enables the system to make informed decisions about task allocation and team formation based on accumulated experience. It directly supports our investigation of how team size influences lifelong learning ability (RQ2) by providing quantitative metrics on collaboration efficiency and specialization emergence.

3.1.2. Memory Topology Configurations

LLMA-Mem supports three fundamental memory topologies that determine knowledge sharing and accessibility patterns across the multi-agent system:

Local Memory Topology In the local configuration, each agent a_i maintains a private memory store $\mathcal{M}_i = \langle \mathcal{E}_i, \mathcal{P}_i, \mathcal{T}_i \rangle$ where $\mathcal{E}_i$, $\mathcal{P}_i$, and $\mathcal{T}_i$ represent episodic, procedural, and team state memories respectively. No direct cross-agent memory access is permitted, though agents may share knowledge through

explicit communication during task execution. This topology maximizes privacy and parallelism but risks knowledge fragmentation and redundant learning.

Shared Memory Topology The shared configuration employs a centralized memory pool $\mathcal{M}_{\text{shared}} = \langle \mathcal{E}_{\text{shared}}, \mathcal{P}_{\text{shared}}, \mathcal{T}_{\text{shared}} \rangle$ accessible to all agents in the system. Every experience contributes to collective knowledge, and all agents benefit from the accumulated expertise of the entire team. While this maximizes knowledge sharing, it introduces challenges in conflict resolution, concurrent access control, and memory complexity management as team size scales.

Hybrid Memory Topology The hybrid configuration combines private and shared memory spaces:

$$\mathcal{M}_i^{\text{hybrid}} = \langle \mathcal{E}_i^{\text{local}}, \mathcal{T}_i^{\text{local}}, \mathcal{P}_{\text{shared}}, \mathcal{T}_{\text{shared}} \rangle \tag{6}$$

Episodic memories remain local to preserve agent-specific experiential context, while procedural knowledge and aggregated team statistics are shared. This design balances knowledge sharing with privacy, reducing the memory complexity burden on individual agents while enabling collective benefit from extracted procedures. Local team state $\mathcal{T}_i^{\text{local}}$ captures an agent's subjective view of collaboration dynamics, while $\mathcal{T}_{\text{shared}}$ maintains objective team-level statistics.

3.1.3. Memory Lifecycle

The memory lifecycle in LLMA-Mem consists of four phases that govern how knowledge is created, accessed, updated, and refined over the system's operational lifetime.

Memory Initialization At system initialization ($t = 0$), all memory stores are empty: $\mathcal{E}^{(0)} = \emptyset$, $\mathcal{P}^{(0)} = \emptyset$, $\mathcal{T}^{(0)} = \emptyset$. Optionally, a small seed set of domain-general procedures $\mathcal{P}_{\text{seed}}$ may be provided to bootstrap early performance, though our experiments evaluate true tabula rasa learning without pre-seeded knowledge.

Memory Retrieval During task execution, agents retrieve relevant knowledge through a multi-factor scoring function. For a given query context q (e.g., current task description), the relevance score for a memory item m is computed as:

$$\text{score}(m, q) = \lambda_r \cdot \text{recency}(m) + \lambda_v \cdot \text{relevance}(m, q) + \lambda_i \cdot \text{importance}(m) + \lambda_t \cdot \text{team\_match}(m, q) \tag{7}$$

where the weights $\lambda_r, \lambda_v, \lambda_i, \lambda_t$ sum to 1. We use $\lambda_r = 0.3$, $\lambda_v = 0.4$, $\lambda_i = 0.2$, $\lambda_t = 0.1$ based on preliminary experiments.

The recency component is:

$$\text{recency}(m) = \exp\left(-\gamma \cdot (t_{\text{current}} - t_m)\right) \tag{8}$$

where γ controls the decay rate and t_m is the memory's timestamp.

Relevance is computed using semantic similarity:

$$\text{relevance}(m, q) = \frac{\text{embed}(m) \cdot \text{embed}(q)}{\|\text{embed}(m)\| \|\text{embed}(q)\|} \tag{9}$$

Importance for procedural memory is based on success rate and usage:

$$\text{importance}(p_j) = \rho_j \cdot \log(1 + \text{viewed\_times}_j) \tag{10}$$

Team context matching evaluates whether the memory originated from similar team configurations or collaboration patterns.

The top-k memories with highest scores are retrieved and provided to the agent as context for decision-making.

Memory Update After each task completion, the memory system undergoes updates:

Episodic Addition: A new episode e_{new} is created and appended to episodic memory:

$$\mathcal{E}^{(t+1)} = \mathcal{E}^{(t)} \cup \{e_{\text{new}}\} \tag{11}$$

Procedural Update: For procedures used during the task, success/failure counts are updated using Bayesian statistics:

$$s_j^{(t+1)} = s_j^{(t)} + \mathbb{1}_{\text{success}}, \quad f_j^{(t+1)} = f_j^{(t)} + \mathbb{1}_{\text{failure}} \tag{12}$$

The confidence score is updated as:

$$\sigma_j^{(t+1)} = \frac{s_j^{(t+1)} + \alpha}{s_j^{(t+1)} + f_j^{(t+1)} + 2\alpha} \tag{13}$$

where α is a prior pseudocount (we use $\alpha = 1$).

Team State Update: Agent profiles and team patterns are incrementally updated:

$$\Gamma_k[\tau]^{(t+1)} = \beta \cdot \Gamma_k[\tau]^{(t)} + (1 - \beta) \cdot \text{performance}_{\text{new}} \tag{14}$$

$$\xi_k^{(t+1)} = \frac{\text{successes}_k}{\text{total\_tasks}_k} \tag{15}$$

where $\beta = 0.9$ is a momentum parameter for exponential moving average.

Memory Consolidation Periodically (every N episodes, with $N = 10$ in our implementation), the system performs consolidation to extract procedural knowledge from accumulated episodes. This process identifies recurring successful patterns and crystallizes them into procedures:

[h] [1] **Input:** Episodic memory $\mathcal{E}$, extraction threshold θ_{extract} **Output:** New procedures $\mathcal{P}_{\text{new}}$ Cluster episodes by task similarity using embedding space each cluster C with $|C| \geq \theta_{\text{extract}}$ $E_{\text{success}} \leftarrow$ successful episodes in C $|E_{\text{success}}|/|C| > 0.6$ Extract common action patterns π from E_{success} Identify shared context conditions as prerequisites Create procedure p_{new} with: $\kappa \leftarrow$ generalized strategy from π $\mathcal{E}_{\text{src}} \leftarrow E_{\text{success}}$ $\sigma \leftarrow |E_{\text{success}}|/|C|$ $\mathcal{P}_{\text{new}} \leftarrow \mathcal{P}_{\text{new}} \cup \{p_{\text{new}}\}$ Merge similar existing procedures (cosine similarity > 0.85) Prune low-confidence procedures ($\sigma < 0.3$ and unused for $K = 5$ episodes) $\mathcal{P}_{\text{new}}$

This consolidation mechanism transforms implicit knowledge embedded in raw experiences into explicit, transferable procedures, enabling the system to learn from experience and improve performance over successive tasks—the hallmark of lifelong learning.

3.2. Experimental Design

To address our research questions regarding memory topology's influence on lifelong learning ability (RQ1) and its interaction with team size scaling (RQ2), we conduct systematic experiments across the three memory topologies (local, shared, hybrid) with varying team sizes ranging from $n \in \{1, 2, 4, 8\}$ agents.

Evaluation Metrics We measure lifelong learning ability through:

- Average Task Score (TS) (%) and
- Coordination Score (CS) for Minecraft, Database, Coding, Bargaining, and WereWolf, scores are multiplied by 20. (the TS and CS are provided by origin code of MultiAgentBench)
- **Average Performance (AP):** We additionally adopt *average performance* to summarize overall performance up to time t:

$$\mathrm{AP}_t = \frac{1}{t} \sum_{i=1}^{t} J_{t,i}, \tag{16}$$

  where $J_{t,i}$ denotes the performance (e.g., success indicator or normalized task score) on task i when evaluated after the system has experienced the first t tasks.
- **Average Incremental Performance (AIP):** To capture the historical variation of performance over the entire learning trajectory, we compute *average incremental performance*:

$$\mathrm{AIP} = \frac{1}{T} \sum_{t=1}^{T} \mathrm{AP}_t, \tag{17}$$

  where T is the total number of tasks in the lifelong learning sequence. Compared to AP, AIP better reflects how performance evolves over time rather than only final or aggregated outcomes.

Benchmarks All experiments are conducted over a sequence of $T = 100$ tasks to capture long-term learning dynamics. Each configuration is evaluated across 5 random seeds, and we report mean performance with 95% confidence intervals.

4. Limitations

Scale of team size could be larger.

References

L. Chen, J. Q. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou. Are more llm calls all you need? towards scaling laws of compound inference systems. *arXiv preprint arXiv:2403.02419*, 2024a.

W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *ICLR*, 2024b.

Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*, 2023.

D. Han, C. Couturier, D. M. Diaz, X. Zhang, V. Rühle, and S. Rajmohan. Legomem: Modular procedural memory for multi-agent llm systems for workflow automation. *arXiv preprint arXiv:2510.04851*, 2025.

Y. Kim, K. Gu, C. Park, C. Park, S. Schmidgall, A. A. Heydari, Y. Yan, Z. Zhang, Y. Zhuang, M. Malhotra, et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2025.

J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye. More agents is all you need. *arXiv preprint arXiv:2402.05120*, 2024.

C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. Gonzalez. Memgpt: Towards llms as operating systems. 2023.

J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.

C. Qian, Z. Xie, Y. Wang, W. Liu, K. Zhu, H. Xia, Y. Dang, Z. Du, W. Chen, C. Yang, et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.

S. Rajaram and L. P. Pereira-Pasarin. Collaborative memory: Cognitive research and theory. *Perspectives on psychological science*, 5(6):649–663, 2010.

Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.

S. Wu and K. Shu. Memory in llm-based multi-agent systems: Mechanisms, challenges, and collective intelligence. *Authorea Preprints*, 2025.

W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.

Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Dong, and J.-R. Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43 (6):1–47, 2025.

J. Zheng, C. Shi, X. Cai, Q. Li, D. Zhang, C. Li, D. Yu, and Q. Ma. Lifelong learning of large language model based agents: A roadmap. *arXiv preprint arXiv:2501.07278*, 2025.

K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, D. Z. Wang, Z. Wang, C. Qian, R. Tang, H. Ji, et al. Multiagentbench: Evaluating the collaboration and competition of llm agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8580–8622, 2025.